PyTorch In-Depth Tutorial

From Tensors to Training: Building Neural Networks with PyTorch

1

Part 1: Recap from Last Lecture

2

What We Learned

- **Neural Networks**: Layers of neurons that learn hierarchical features
- **Forward Pass**: Data flows through layers, transformed at each step
- **Backpropagation**: Chain rule enables efficient gradient computation
- **Gradient Descent**: Iteratively update weights to minimize loss

3

The Problem

- Computing gradients by hand for millions of parameters is **impossible**
- Modern models have trillions of parameters (GPT 5.2, Gemini 3 Pro, etc.)
- We need a framework that handles the math **automatically**
- Enter: **PyTorch**

4

Part 2: Why PyTorch?

5

PyTorch Overview

- Open-source ML library by **Meta AI** (Facebook)
- **Two Core Features**:
- 1. Tensor Computing: N-dimensional arrays that run on GPUs
- 2. Automatic Differentiation: Computes gradients for you!
- Dominant framework in research, widely used in industry

6

Why Not Other Frameworks?

- **PyTorch**: Dynamic graphs, Pythonic, easy debugging
- **JAX**: Great for research, steeper learning curve
- **TensorFlow**: Better for production, less popular than PyTorch
- PyTorch strikes the best balance for learning, research and production

7

Part 3: Tensors - The Building Block

8

What is a Tensor?

- A **multi-dimensional array** — generalizes scalars, vectors, matrices
- **0-D tensor**: Scalar (single number)
- **1-D tensor**: Vector (list of numbers)
- **2-D tensor**: Matrix (table of numbers)
- **3-D tensor**: Cube (e.g., RGB image: channels × height × width)

🚀 **Interactive Demo:** tensor_demo.html

9

Creating Tensors

PYTHON

```python
import torch

# From a Python list
data = [[1, 2], [3, 4]]
x = torch.tensor(data)

# Tensors of ones, zeros, random
ones = torch.ones(2, 3)
zeros = torch.zeros(2, 3)
rands = torch.rand(2, 3)

# From NumPy
import numpy as np
np_array = np.array([1, 2, 3])
x_np = torch.from_numpy(np_array)
```

Tensor Operations

PYTHON

```python
# Element-wise operations
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

print(a + b)      # tensor([5, 7, 9])
print(a * b)      # tensor([4, 10, 18])
print(a ** 2)     # tensor([1, 4, 9])

# Matrix multiplication
A = torch.rand(2, 3)
B = torch.rand(3, 4)
C = A @ B         # Shape: (2, 4)
```

Tensor Attributes

PYTHON

```python
x = torch.rand(3, 4)

print(x.shape)    # torch.Size([3, 4])
print(x.dtype)    # torch.float32
print(x.device)   # cpu or cuda:0
```

- **Shape**: Dimensions of the tensor
- **Dtype**: Data type (float32, float64, int64, etc.)
- **Device**: Where the tensor lives (CPU or GPU)

12

Part 4: Autograd - Automatic Differentiation

13

The Magic of requires_grad

- Set `requires_grad=True` to track all operations on a tensor
- PyTorch builds a **computational graph** behind the scenes
- Call `.backward()` to compute all gradients automatically
- Access gradients via `.grad` attribute

🚀 **Interactive Demo:** autograd_demo.html

14

Autograd Example

PYTHON

```python
# Create tensors with gradient tracking
w = torch.tensor(2.0, requires_grad=True)
x = torch.tensor(3.0)

# Forward pass builds the graph
y = w * x  # y = 2 * 3 = 6

# Backward pass computes gradients
y.backward()

# Gradient is dy/dw = x = 3
print(w.grad)  # tensor(3.)
```

15

Key Autograd Concepts

- **Computational Graph**: Tracks operations for automatic differentiation
- **Leaf Tensors**: Tensors created by the user (have `.grad`)
- **grad_fn**: Each tensor stores how it was created (for backprop)
- **Gradients accumulate** by default — use `optimizer.zero_grad()`!

16

Part 5: Building Networks with torch.nn

17

The nn.Module Class

- **nn.Module**: Base class for all neural network modules
- Define your network by creating a class that inherits from `nn.Module`
- Two methods to implement:
- 1. `__init__()`: Define the layers
- 2. `forward()`: Define how data flows through layers

18

Simple Neural Network

PYTHON

```python
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 128)
        self.layer2 = nn.Linear(128, 64)
        self.layer3 = nn.Linear(64, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.layer1(x))
        x = self.relu(self.layer2(x))
        x = self.layer3(x)
        return x
```

19

Common Layers

- **nn.Linear(in, out)**: Fully connected layer, computes $y = xW^T + b$
- **nn.ReLU()**: ReLU activation function
- **nn.Sigmoid()**: Sigmoid activation function
- **nn.Softmax(dim)**: Softmax for multi-class classification
- **nn.Dropout(p)**: Randomly zero elements during training (regularization)
- **nn.BatchNorm1d(features)**: Batch normalization

20

Part 6: Loss Functions & Optimizers

21

Loss Functions

- **nn.MSELoss()**: Mean Squared Error — for regression

  PYTHON

  ```python
  loss_fn = nn.MSELoss()
  loss = loss_fn(predictions, targets)
  ```

- **nn.CrossEntropyLoss()**: For classification (includes Softmax)

  PYTHON

  ```python
  loss_fn = nn.CrossEntropyLoss()
  loss = loss_fn(logits, labels)  # labels are class indices
  ```

22

Optimizers

PYTHON

```
import torch.optim as optim

# SGD with momentum
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9

# Adam (most popular)
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

- Optimizers update model weights based on computed gradients

23

The Three Sacred Lines

PYTHON

```python
# 1. Clear old gradients
optimizer.zero_grad()

# 2. Compute gradients via backpropagation
loss.backward()

# 3. Update weights
optimizer.step()
```

- These three lines are in **every** training loop!

24

Part 7: Dataset & DataLoader

25

Custom Dataset Class

PYTHON

```python
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]
```

26

Using Built-in Datasets

PYTHON

```python
from torchvision import datasets, transforms

# Define transformations
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Load FashionMNIST
train_data = datasets.FashionMNIST(
    root='data', train=True, download=True,
    transform=transform
)
```

DataLoader

PYTHON

```python
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_data,
    batch_size=64,
    shuffle=True,
    num_workers=4
)

# Iterate over batches
for images, labels in train_loader:
    # images: (64, 1, 28, 28)
    # labels: (64,)
    pass
```

28

Part 8: The Full Training Loop

29

Training Recipe (Pseudocode)

- **1. Load Data**: Prepare Dataset and DataLoader
- **2. Define Model**: Create your nn.Module
- **3. Define Loss & Optimizer**: Choose appropriate functions
- **4. Training Loop**:
- - For each epoch:
- - For each batch:
- - Forward pass
- - Compute loss
- - Backward pass (compute gradients)
- - Update weights

30

Complete Training Loop

PYTHON

```python
model = SimpleNet()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    for images, labels in train_loader:
        # Flatten images
        images = images.view(images.size(0), -1)

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward pass & update
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f'Epoch {epoch+1}, Loss: {loss.item():.4f}')
```

Part 9: Evaluation & Testing

32

Evaluation Mode

- `model.eval()` — Sets model to evaluation mode:
- - Disables Dropout
- - Uses running stats for BatchNorm
- `model.train()` — Returns to training mode
- `torch.no_grad()` — Disables gradient computation (saves memory)

33

Evaluation Loop

PYTHON

```python
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images = images.view(images.size(0), -1)
        outputs = model(images)

        # Get predictions
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f'Test Accuracy: {accuracy:.2f}%')
```

34

Part 10: GPU Acceleration

35

Moving to GPU

PYTHON

```python
# Check CUDA availability
device = torch.device('cuda' if torch.cuda.is_available() else
print(f'Using device: {device}')

# Move model to GPU
model = SimpleNet().to(device)

# Move data to GPU in training loop
for images, labels in train_loader:
    images = images.to(device)
    labels = labels.to(device)
    # ... rest of training
```

36

GPU Best Practices

- Always use the same device for model and data
- Use `.to(device)` pattern for portability
- Batch operations are **much faster** on GPU
- Watch out for GPU memory — clear cache if needed

37

Part 11: Saving & Loading Models

38

Saving Models

PYTHON

```python
# Save entire model (not recommended)
torch.save(model, 'model.pth')

# Save only state dict (recommended)
torch.save(model.state_dict(), 'model_weights.pth')

# Save checkpoint (for resuming training)
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': loss,
}
torch.save(checkpoint, 'checkpoint.pth')
```

Loading Models

PYTHON

```python
# Load state dict
model = SimpleNet()
model.load_state_dict(torch.load('model_weights.pth'))
model.eval()

# Load checkpoint
checkpoint = torch.load('checkpoint.pth')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
epoch = checkpoint['epoch']
```

Same Building Blocks

- All modern architectures use the same fundamentals:
- - Tensors
- - Autograd
- - nn.Module
- - Forward/Backward passes
- - Optimizers
- Master these, and you can build anything! 🚀

41

Key Takeaways

- **Tensors**: Multi-dimensional arrays that power PyTorch
- **Autograd**: Automatic differentiation via computational graphs
- **nn.Module**: Base class for building network architectures
- **Training Loop**: zero_grad → forward → loss → backward → step
- **Evaluation**: model.eval() + torch.no_grad()

42